

Checked Semantic Equality Saturation for Finite Boolean Programs

An Exploratory ZenoFCIS Case Study

Dana Edwards

Draft — October 3, 2026

Abstract

We evaluate exact Boolean truth signatures as an additional source of equivalence in an e-graph optimizer for ZenoFCIS’s finite program representation. A fixed synthetic corpus contains 100 cases with three to six Boolean inputs and ordered, sometimes repeated, outputs. A strong local simplifier already performs factoring, absorption, and structural sharing. Two initial resource configurations fail because integer extraction reaches solver limits. A smaller, exploratorily chosen search configuration passes all checks and reduces the aggregate emitted instruction count from 895 for local simplification to 816, an 8.8% reduction. It improves 18 cases across three families, ties 78, and regresses on four. Compared with otherwise identical rewrite-only egg, semantic merging saves 38 instructions and improves 12 cases without regressions. Every candidate is checked over its entire admitted Boolean domain through the actual evaluator and separate expression and serialized-program interpreters. A Lean model proves complete-signature and acceptance-gate properties; it does not verify the implementation. These results support a bounded optimization case study, with no claim of application runtime speedup or Tau integration.

1 Question and contribution

Equality saturation retains alternative equivalent programs before choosing an implementation, reducing dependence on destructive rewrite order [1, 2]. Domain semantics can supply equalities that a bounded syntactic search misses. We ask a narrower engineering question: can complete finite Boolean signatures improve shared programs in ZenoFCIS beyond a strong local simplifier and the same e-graph using ordinary rewrites alone?

The contribution is a reproducible, independently checked case study with explicit failure records and proof boundaries. Truth tables, Boolean circuit optimization, and semantic e-graphs are established techniques [3, 4, 5, 6]. We do not introduce a new Boolean-equivalence algorithm, compare against mature logic synthesis tools, or establish a globally optimal extracted program.

The experiment uses the pinned `zeno-fcis/finite-i64/1` program evaluator [8]. It operates offline on a total Boolean fragment containing inputs, constants, negation, conjunction, disjunction, and conditionals. No production execution path is changed.

2 Finite semantics and the acceptance boundary

Let $\mathbb{B} = \{0, 1\}$ and $D_n = \mathbb{B}^n$, with $0 \leq n \leq 6$ for the implementation. Fix one enumeration $d_0, \dots, d_{2^n-1}$, where bit i of a row mask supplies input v_i . For a Boolean expression e , define

$$\text{sig}_n(e) = (\llbracket e \rrbracket(d_0), \dots, \llbracket e \rrbracket(d_{2^n-1})).$$

The implementation packs this complete signature into a `u64`; it uses exact bit-vector equality rather than a probabilistic semantic hash. Conjunction and disjunction are bitwise operations, and negation is masked to the admitted rows. Both egg variants compute this analysis as a consistency check. Only the semantic variant additionally unions classes with equal signatures.

Proposition 1. *For total Boolean expressions over the fixed interface, $\text{sig}_n(e) = \text{sig}_n(f)$ if and only if $\forall d \in D_n, \llbracket e \rrbracket(d) = \llbracket f \rrbracket(d)$.*

Proof. Every admitted valuation appears in the same enumeration for both signatures. Equality of lists therefore gives equality at each valuation, and pointwise equality gives equality of lists. $\square$

There are 2^n valuations and 2^{2^n} single-output Boolean functions. For m ordered Boolean outputs, the number of functions is 2^{m2^n} . These finite counts concern a fixed semantic interface. They do not make syntax finite, bound the number of alternative e-nodes stored during rewriting, or guarantee a practical search size.

For an actual program P , a more suitable observation retains every output position and every error:

$$\text{Obs}(P, d) \in \mathbb{B}^m + \mathcal{E}, \quad P \equiv_D Q \iff \forall d \in D, \text{Obs}(P, d) = \text{Obs}(Q, d).$$

The runtime gate compares ordered output tuples or the actual distinct error, not merely contract satisfaction or successful-return values. It also checks that input and output schemas agree. All emitted Boolean programs are admitted through ZenoFCIS’s constructor, which checks types, shape, and references.

A separate protected-selection simulation uses the checked local baseline P as fallback. Let $\text{ToolChecks}(Q)$ require a positive optimal solver status and no optimizer failure, panic, or saturation-watchdog timeout. For a proposal Q , define

$$A(P, Q) = \text{Valid}(Q) \wedge \text{ToolChecks}(Q) \wedge P \equiv_{D_n} Q \wedge \text{nodes}(Q) < \text{nodes}(P).$$

Its states are

$$\text{Baseline}(P) \xrightarrow{\text{search}} \text{Proposal}(P, Q) \xrightarrow{\text{check}} \begin{cases} \text{Selected}(Q), & A(P, Q), \\ \text{Selected}(P), & \neg A(P, Q). \end{cases}$$

Solver failures remain failures of the experiment even when a policy could fall back. Raw proposal regressions remain visible.

2.1 Why errors and source identity matter

ZenoFCIS evaluates all instructions eagerly. Removing an unused overflowing addition changes an arithmetic error into success. An unselected conditional arm can likewise overflow before the conditional instruction executes. The optimizer conservatively excludes arithmetic and comparison nodes, including dead nodes and conditional arms. Negative controls explicitly reject these unsafe changes.

Observational equality here does not preserve a program’s canonical bytes or checksum. Changing the instruction DAG can also change resource accounting. This pilot does not establish equivalence of authoritative state transitions, events, effects, receipts, budgets, or an imperative shell.

3 Experimental method

3.1 Corpus and local baseline

The corpus was generated before optimizer results with Python 3.9.6 and seed 20261003. It contains 100 cases, with input arities distributed as 18 cases at three inputs, 29 at four, 27 at five, and 26 at six. There are 76 single-output and 24 multiple-output cases. Output order and duplicate positions are significant. No case is filtered according to results. The families and all aggregate outcomes appear in Table 2. The general family uses seeded balanced expression trees of depths three and four; the other families are constructed identity and sharing patterns.

The deterministic local baseline proposes constant folding, idempotence, complements, absorption, double negation, associative normalization, De Morgan identities, conditional simplifications, and common factoring. A replacement changes all occurrences of a structural subexpression across all roots. At each step it selects the best strict lexicographic decrease in

(shared lowered node count, total expression length, ordered expression spellings).

Proposals do not consult truth tables. Exhaustive enumeration verifies the result separately. The 1,024-step ceiling is enforced; this corpus needs at most 12 steps. Identical subexpressions are shared in every variant, including the raw and local programs.

3.2 E-graph search and cost

Both egg variants start from the raw outputs plus the same constants and declared input leaves. They use the same 33 ordinary rules, scheduler, resource limits, and shared-DAG integer extractor [9]. The rules include factoring, De Morgan identities, conditional identities, commutation, and reassociation. Semantic merging adds exact-signature union sweeps during search and before extraction. Binary egg rules expose generalized factors through bounded commutative and associative exploration; the local baseline instead considers flattened factors directly. These are different search policies.

The final representation uses `Input`, `Bool`, `And`, `Not`, and `Select`. Disjunction lowers as

$$a \vee b = \neg(\neg a \wedge \neg b).$$

The extractor assigns cost one to each operator except disjunction, whose surrogate cost is four. Generated negations can be shared after lowering, so the additive surrogate differs from final emitted DAG cost. All primary measurements count actual distinct emitted instructions across every output. An optimal backend status is optimality reported for the constructed class-consistent surrogate model, subject to numerical tolerances. It is not a proof of minimum emitted cost over all equivalent programs.

3.3 Resource revisions and checks

The cost criterion was fixed before the first full run: all tool and verification checks must pass; semantic proposals must have fewer aggregate instructions than the local baseline, improve at least ten cases, and improve at least two families. The initial resource configuration fails this criterion. Table 1 records all full configurations. Later resource choices are exploratory revisions after observing failures; the original registered configuration is not reported as successful.

The first run has two rewrite-only and ten semantic time-limit failures. Nine cannot be decoded into selected integer DAGs; three produce behaviorally correct incumbents that remain rejected for

Iterations	Node threshold	CBC seconds	Failed extractions	Outcome
6	5,000	2	12	Tool gate failed
6	5,000	30	1	Tool gate failed
3	1,000	30	0	All gates passed
3	1,000	30	0	Repeat passed

Table 1: Every full configuration uses the same 100 cases, rules, objective, and correctness requirements. Node thresholds are checked between batches and can be exceeded within a batch.

solver status. At 30 seconds, one semantic extraction still times out. Its 22-node incumbent passes behavior checks against a 30-node local baseline but is not accepted. The smaller-search configuration then passes all 200 positive solver-status checks. A 600-second emergency saturation watchdog would also fail a run.

For each successful full configuration, all 400 variants undergo 12,544 complete-domain candidate/input comparisons. Separate Python expression evaluation, Rust expression evaluation, actual `Program.evaluate`, and a Python interpreter for the serialized instruction DAG agree. Another 8,176 invalid-input probes check both wrong-length boundaries and each input position at -1 , 2 , and the signed 64-bit extremes. These probes sample the infinite invalid-input space; they are not its exhaustive enumeration.

Six controls check altered truth functions, a non-Boolean region, unused arithmetic overflow, unselected-arm overflow, invalid input shape, and invalid input domain. All pass. Three focused harness tests and seven existing ZenoFCIS finite-module tests pass. The result analyzer also rejects 13 seeded receipt defects. A repeat has identical candidate expressions, emitted programs, signatures, observations, selection decisions, and search snapshot metadata; timing fields differ.

4 Results

Family	Cases	Raw	Local	Rewrite	Semantic	Wins/local
Small gates	8	34	18	18	18	0
Distributed factoring	18	217	169	170	170	0
Absorption/complement	18	159	48	42	42	1
Conditional/De Morgan	18	157	120	124	120	0
Shared guards	18	357	246	243	243	6
General balanced	20	582	294	257	223	11
Total	100	1,506	895	854	816	18

Table 2: Actual shared emitted instruction counts for the successful bounded configuration. Wins are strict per-case improvements, not differences of family totals.

Semantic proposals use 816 instructions versus 895 for the local baseline: an 8.8% aggregate reduction, with 18 wins, 78 ties, and four regressions. The winning families are absorption/complement, shared guards, and general balanced expressions. All four regressions add one instruction each. The protected selection therefore uses 812 instructions, choosing semantic proposals in 18 cases and the local baseline in 82. The 812-node policy result is separate from the primary 816-node raw proposal result.

Rewrite-only egg uses 854 instructions, with 13 wins, 80 ties, and seven regressions against

local simplification. Adding semantic unions improves 12 cases, ties 88, and regresses on none relative to rewrite-only egg. Its 38-instruction additional reduction is 4.4%. This paired comparison supports an effect from semantic merging under the evaluated configuration. Much of the overall improvement over local simplification also comes from ordinary e-graph search.

For example, `balanced_general-08` has local outputs

$$(\neg v_3, \neg \text{ite}(v_3, v_2 \vee v_3, \neg v_5)).$$

When $v_3 = 1$, the second component is false; when $v_3 = 0$, it is v_5 . Thus the ordered outputs equal $(\neg v_3, \neg v_3 \wedge v_5)$. The raw, local, rewrite-only, and semantic programs use 26, 10, 7, and four instructions respectively. The final four are input v_3 , its negation, input v_5 , and their conjunction, with the negation shared between outputs. The six-input schema remains unchanged and all 64 valuations are checked. This illustrative case is included in the full aggregates.

4.1 Timing and graph size

The host is an Apple M3 Max with 16 CPU cores and 128 GB RAM, running macOS 26.6.2. CBC is configured with one thread and both random seeds set to one. The table reports optimization intervals, comprising graph construction/search, extraction, and reconstruction; verification is measured separately.

Variant	First bounded run		Repeat	
	Sum (ms)	Median (ms)	Sum (ms)	Median (ms)
Rewrite	10,530.7	1.679	510.7	1.547
Semantic	761.1	1.572	763.2	1.460

Table 3: Descriptive optimization timings over 100 cases. The initial rewrite sum contains a 10.0-second outlier. These measurements do not establish a stable speedup.

The harness-recorded case-processing intervals are 11.395 seconds and 1.368 seconds. They include candidate checks and receipt construction, but exclude corpus loading, the separate controls, and final JSON encoding and file writes. The timing reversal precludes a stable aggregate optimizer-speed claim. Separately measured Python baseline search has a 0.146 ms median case over three repetitions and 59.834 ms sum of case medians. Different languages and measurement scopes preclude a fair algorithmic speed ratio. Application execution throughput is not benchmarked.

The largest bounded graphs contain 560 rewrite-only and 1,351 semantic e-nodes. Semantic unions can expose additional rewrite matches; they do not guarantee smaller graph storage. The 1,351-node graph also demonstrates batch overshoot of the configured 1,000-node threshold. Bounded search is not necessarily full saturation.

5 Formal checking and relation to Tau

The accompanying standalone Lean model is replayed with Lean 4.30.0-rc2 and Mathlib revision 9977002. It checks finite valuation and function counts, equality of complete signatures if and only if pointwise equality, congruence for total Boolean contexts including conditionals, distributive factoring, and a candidate-selection theorem. The latter permits an arbitrary generator and additional acceptance condition while preserving the entire observation, including an error or every ordered output.

The seven reported theorem closures use only the standard foundations `propext`, `Classical.choice`, and `Quot.sound`. There are no proof placeholders, new axiom declarations, or native-decision escapes. The model’s finite enumeration is mathematical and noncomputable; it is not a refinement proof of the Rust mask loop. Rust semantics, lowering, e-class maintenance, CBC, solver numerical behavior, and resource or canonical identity preservation remain outside this formal proof.

Asor describes a broader finite quotient for atomless Boolean-algebra formulas with fixed finite free-variable and constant scope; quantifier elimination extends that result to quantified formulas [7]. This pilot uses only ordinary finite Boolean inputs. It does not implement that quantified theory, sentence-valued data, guarded successor, or a Tau decision procedure. A future Tau application would need an appropriate equivalence oracle and context-preservation argument. Temporal recurrence additionally requires appropriate recurrence-state and fixed-point conditions; finite quotients alone do not imply that every recurrence stabilizes rather than cycles.

6 Limits and next experiments

The corpus is synthetic, partly constructed around identities, small, and uses only one seeded general-tree collection. It does not represent an application distribution. Resources were revised after observed failures; the result is exploratory rather than confirmation under an untouched registered protocol. The benefit is against this local baseline and rewrite set, not ABC, BDD, Espresso, BoolE, or Nextmap. The cost model counts all instructions uniformly and does not measure hardware area, application throughput, or protocol-visible budgets. Solver-reported surrogate optimality cannot eliminate the four regressions in final emitted cost.

The next useful tests are held-out application guards, comparison with mature Boolean synthesis, better alignment between extraction and lowered sharing, and end-to-end verification that includes surrounding eager producers and the actual resource contract. The present evidence supports smaller, behaviorally checked Boolean proposals within the stated scope.

A Artifact identity and reproduction

The companion source-and-evidence directory contains the generator, frozen corpus, locked Rust harness, exact proposals and observations, independent analysis, failed-run audits, resource amendments, repeat receipt, and Lean source with replay receipt. The directory and this paper draft are available at <https://github.com/TheDarkLightX/ZenoFCIS/tree/research/semantic-egraphs/experiments/semantic-egraphs>.

Place the experiment at `experiments/semantic-egraphs` in the pinned ZenoFCIS checkout. Use Rust 1.97.1, a linkable CBC 2.10.13, the experiment’s `Cargo.lock`, and the pinned egg source. The successful configuration is:

```
cargo +1.97.1 run --release --locked \
  --manifest-path experiments/semantic-egraphs/Cargo.toml -- \
  experiments/semantic-egraphs/corpus.json results.json \
  --iterations 3 --node-limit 1000 --lp-seconds 30
python3 experiments/semantic-egraphs/analyze.py results.json \
  --out analysis.json --markdown analysis.md
```

The optional Lean replay uses an existing matching Mathlib project:

```
bash proof/replay.sh /path/to/matching/mathlib-project
```

Run that command from the experiment directory. The original failed configurations, including commands and audits, remain in its README.

Exact source pins are:

- ZenoFCIS: `cb1366cddc5c813f9977b377f541e64bcf94035c`.
- egg: `12997cd15a123156abd726155363701f13358040`.
- Mathlib: `9977002c3c9492b622fb469b0d18acc7e73aed3e`.

The frozen corpus SHA-256 is

`f0ad16732a453ef6c176e48aa46e544ad07015f4793bf32c84459a271ee9bc18`.

The Lean source SHA-256 is

`d4b6aeed147e2d6f769342cabe4cda491f96006205f49d1014836a9c4a9af6c`.

References

- [1] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. *Equality Saturation: A New Approach to Optimization*. POPL, 2009. <https://www.cs.cornell.edu/~lerner/papers/pop109.html>.
- [2] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. *egg: Fast and Extensible Equality Saturation*. Proceedings of the ACM on Programming Languages 5, POPL, Article 23, 2021. <https://doi.org/10.1145/3434304>.
- [3] Randal E. Bryant. *Graph-Based Algorithms for Boolean Function Manipulation*. IEEE Transactions on Computers C-35(8), 677–691, 1986. <https://doi.org/10.1109/TC.1986.1676819>.
- [4] Alan Mishchenko, Satrajit Chatterjee, and Robert Brayton. *DAG-Aware AIG Rewriting: A Fresh Look at Combinational Logic Synthesis*. DAC, 2006. https://people.eecs.berkeley.edu/~alanmi/publications/2006/dac06_rwr.pdf.
- [5] Jiaqi Yin, Zhan Song, Chen Chen, Qihao Hu, and Cunxi Yu. *BoolE: Exact Symbolic Reasoning via Boolean Equality Saturation*. DAC, 2025; arXiv:2504.05577. <https://arxiv.org/abs/2504.05577>.
- [6] Sijie Kong, Jingtao Xia, Daniel Ruelas-Petrisko, Zachary D. Sisco, Jonathan Balkind, and Gus Henry Smith. *Improving Equality Saturation for EDA via Semantic E-Graphs*. Proceedings of the ACM on Programming Languages 10, PLDI, Article 221, 2026. <https://doi.org/10.1145/3808299>.
- [7] Ohad Asor. *Guarded Successor: A Novel Temporal Logic*. arXiv:2407.06214v1, 2024, Sections 2.3–2.4. <https://arxiv.org/html/2407.06214v1>.
- [8] ZenoFCIS source repository, revision `cb1366c`. Finite-program evaluator and representation. <https://github.com/TheDarkLightX/ZenoFCIS>.
- [9] egg source repository, revision `12997cd`. LP extraction implementation used in this experiment. <https://github.com/TheDarkLightX/egg>.